

DAST Integration Guide

Integrating OWASP ZAP into a Jenkins CI/CD Pipeline

Project: webapp-cicd-pipeline

1. Overview

This document describes how Dynamic Application Security Testing (DAST) was added to an existing Jenkins CI/CD pipeline, using OWASP ZAP (Zed Attack Proxy). Unlike SAST (Semgrep) and SCA (Snyk), which analyze source code and dependencies before the application is built, DAST scans the application while it is actually running — after deployment — by sending real HTTP requests to it and observing the responses.

The complete pipeline flow, in order, is:

1. Secret Scan (TruffleHog) – scans the Git repository for hardcoded secrets.
2. SCA Scan (Snyk) – scans project dependencies (pom.xml) for known vulnerable libraries.
3. SAST Scan (Semgrep) – scans the Java source code for insecure coding patterns.
4. Build (Maven) – compiles the application and packages it as a WAR file.
5. Deploy to Tomcat – copies the WAR file to the Tomcat application server.
6. DAST Scan (OWASP ZAP) – scans the live, deployed application for runtime vulnerabilities.

A key design decision for this pipeline is that none of the security stages block the build. Every scan stage marks the Jenkins build as UNSTABLE (not FAILURE) when issues are found, and the pipeline always continues through Build, Deploy, and DAST. This was intentional, to allow the team to see and track all findings across every layer of security testing in a single build, rather than stopping at the first failure.

2. Environment / Architecture

The setup uses two separate AWS EC2 instances:

- Jenkins server — runs the pipeline, Semgrep, Snyk, TruffleHog, and OWASP ZAP.
- Tomcat server — hosts the deployed web application (WebApp.war).

Because Jenkins and Tomcat are two different EC2 instances, OWASP ZAP (running on the Jenkins server) has to reach the Tomcat server over the network using its address — not "localhost". This distinction turned out to be the main source of connectivity issues during setup, covered in Section 4.

explicitly allowed, which caused ZAP to repeatedly fail with a connection timeout when it tried to scan the deployed application.

4.1 Rule required: Tomcat instance (inbound)

On the Security Group attached to the Tomcat EC2 instance, an inbound rule was added to allow traffic from the Jenkins server on the Tomcat application port:

Type	Protocol	Port	Source	Purpose
Custom TCP	TCP	8080	172.31.8.38/32 (Jenkins private IP)	Allows Jenkins / ZAP to reach the deployed WebApp
SSH	TCP	22	0.0.0.0/0 (or admin IP only)	Allows SSH access for deployment / troubleshooting

4.2 Rule required: Jenkins instance (outbound)

On the Security Group attached to the Jenkins EC2 instance, the default outbound rule (present on new AWS security groups) was kept as-is, since it already permits all outbound traffic:

Type	Protocol	Port	Destination	Purpose
All traffic	All	All	0.0.0.0/0	Default outbound rule — allows Jenkins/ZAP to initiate connections to Tomcat and the internet

4.3 Important prerequisite: same VPC

For the private-IP based rule above to work at all, both EC2 instances must be in the same VPC (and ideally the same subnet / availability zone). Private IP addresses are only directly routable within the same VPC; a private IP from one VPC cannot be reached from an instance in a different VPC without VPC peering or a similar setup. Both instances in this project were confirmed to be in the same VPC before the rule was added.

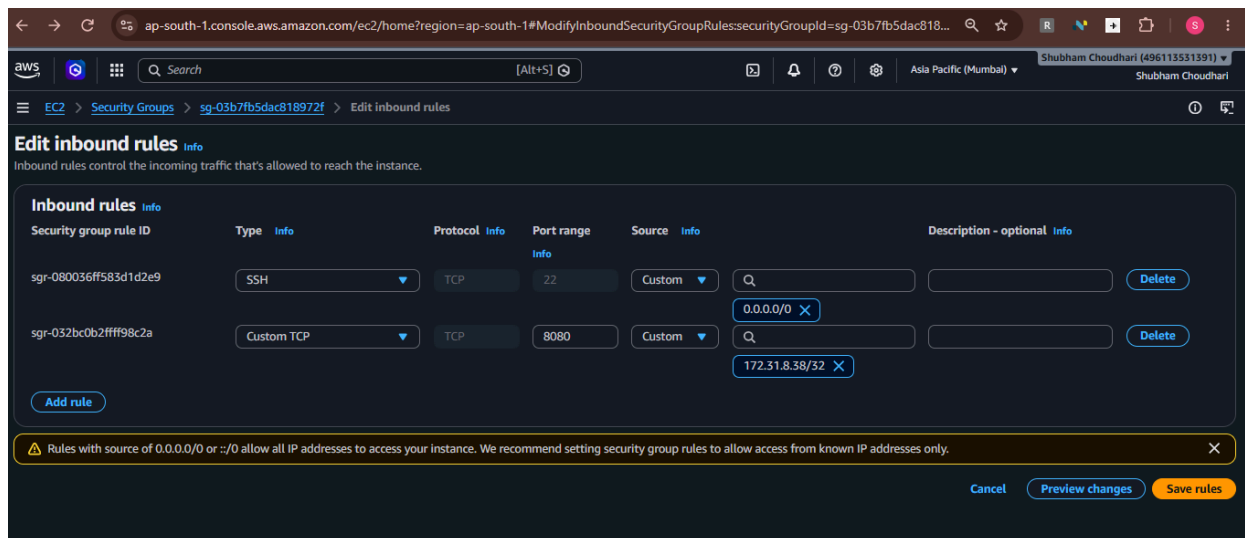


Figure 2 — Inbound rules on the Tomcat instance's Security Group: SSH (22) open, and Custom TCP (8080) restricted to the Jenkins server's private IP (172.31.8.38/32).

4.4 Symptom before the fix

Before this rule was added, running the DAST stage produced the following in the ZAP output, even though the same URL worked perfectly when opened from the Tomcat server itself (localhost):

```
Failed to attack the URL: Connect to http://13.204.66.212:8080 [/13.204.66.212] failed: Connect timed out
```

This is the classic signature of a Security Group block: the TCP connection attempt is silently dropped rather than actively refused, so the client waits for the full timeout period before giving up.

5. Jenkinsfile Integration — DAST Stage

The DAST stage was added as the final stage of the pipeline, placed immediately after "Deploy To Tomcat", since ZAP needs the application to already be running before it can scan it.

5.1 Pipeline environment variables

```
environment {
    SEMGREP_PATH = "/home/ubuntu/.local/bin"
    SNYK_PATH     = "/usr/local/bin"
    ZAP_PATH      = "/opt/zaproxy"
    DEPLOY_HOST   = "13.204.66.212"
    APP_URL       = "http://13.204.66.212:8080/WebApp/"
}
```

5.2 DAST stage

```
stage('DAST Scan - OWASP ZAP') {
    steps {
        script {
            sleep 25

            // Kill any stray ZAP process left over from a previous run
            sh 'pkill -f zap.sh || true'
            sh 'sleep 3'
```

```
sh """
    ${ZAP_PATH}/zap.sh -cmd \
        -port 8090 \
        -quickurl ${APP_URL} \
        -quickprogress \
        -quickout \${WORKSPACE}/zap-report.html || true
"""

archiveArtifacts artifacts: 'zap-report.html', allowEmptyArchive: true

if (fileExists('zap-report.html')) {
    def zapReport = readFile('zap-report.html')
    def highCount = zapReport.count('Risk Level: High')

    if (highCount > 0) {
        unstable "OWASP ZAP found ${highCount} High-risk vulnerabilities"
    } else {
        echo "No High-risk alerts found by ZAP."
    }
}
}
```

5.3 What each part does

- `sleep 25` — gives Tomcat time to fully load the newly deployed WAR file before it is scanned.
- `pkill -f zap.sh || true` — kills any ZAP process left running from a previous (possibly failed) build, so it does not hold onto a port.
- `-port 8090` — tells ZAP to run its internal proxy on port 8090 instead of its default port 8080. This was required because Jenkins itself runs on port 8080 on the same server, and ZAP's default port clashed with it, causing a `java.net.BindException: Address already in use error`.
- `-quickurl ${APP_URL}` — runs ZAP's Quick Scan (spider + active scan) against the deployed application URL.
- `-quickout` — writes the scan results to an HTML report (`zap-report.html`) inside the Jenkins workspace.
- `archiveArtifacts` — makes the report downloadable from the Jenkins build page (Artifacts tab).
- The High-risk count check — a lightweight way of flagging serious findings in the Jenkins console without failing the build; the report itself contains the full detail.

As with the other scan stages (Secret Scan, SCA, SAST), this stage never fails the build outright — it only marks it UNSTABLE — so that Build and Deploy always complete and the DAST report is always produced for review.

6. Issues Encountered and Fixes

A short summary of the real issues hit while wiring this up, for future reference:

Symptom	Root Cause	Fix
ZAP failed to start: java.net.BindException: Address already in use	ZAP's default proxy port (8080) is the same port Jenkins itself runs on, on the same server	Explicitly set ZAP to use a different port with -port 8090
ZAP: "Failed to attack the URL ... Connect timed out"	AWS Security Group on the Tomcat instance did not allow inbound	Added an inbound rule for port 8080, source = Jenkins private IP

Symptom	Root Cause	Fix
	traffic on port 8080 from the Jenkins server's IP	
curl from the Jenkins server to the Tomcat public IP also timed out	Same Security Group restriction — confirmed the block was independent of any local SSH tunnel / browser setup	Same fix as above; verified with curl -I from inside the Jenkins server after the rule was added
Jenkins UI itself became unreachable at one point	The SSH command used to reach the Jenkins server was missing the -L 8080:localhost:8080 port-forwarding flag	Reconnected with the correct SSH command including -L 8080:localhost:8080

7. Pipeline Execution Result

Once the port conflict and the Security Group rule were both fixed, the DAST stage completed a full ZAP Quick Scan (spider + active scan, 0% to 100%) against the live application and produced a report — all without failing the pipeline.

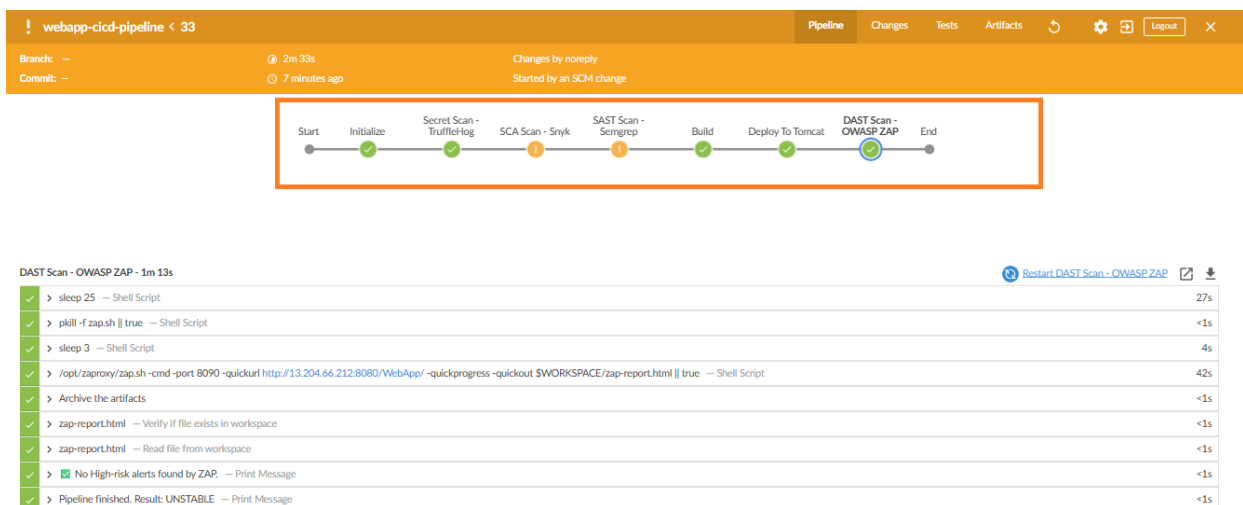


Figure 3 — Jenkins Blue Ocean view of the completed pipeline. All stages, including "DAST Scan - OWASP ZAP", finished successfully; the build overall was marked UNSTABLE because earlier scan stages (SCA, SAST) found issues — by design, this does not block Build or Deploy.

8. ZAP DAST Scan Report Summary

The generated zap-report.html is attached to every Jenkins build under the Artifacts tab, and can also be opened directly at a URL of the form:

<http://localhost:8080/job/webapp-cicd-pipeline/<build-number>/artifact/zap-report.html>

Summary of alerts from the scan of the deployed application:

Name	Risk Level	Number of Instances
Cross Site Scripting (Persistent)	High	2
Cross Site Scripting (Reflected)	High	8
External Redirect	High	1
Open Redirect	High	1
Path Traversal	High	1
Remote File Inclusion	High	1
SQL Injection	High	5
Absence of Anti-CSRF Tokens	Medium	2
Content Security Policy (CSP) Header Not Set	Medium	19
Missing Anti-clickjacking Header	Medium	15
XSLT Injection	Medium	1
Cookie without SameSite Attribute	Low	1
Cross-Domain JavaScript Source File Inclusion	Low	8
X-Content-Type-Options Header Missing	Low	17
Authentication Request Identified	Informational	1
Charset Mismatch (Header Versus Meta Charset)	Informational	2
Information Disclosure - Suspicious Comments	Informational	2
Modern Web Application	Informational	2
Session Management Response Identified	Informational	1
User Agent Fuzzer	Informational	48

Alert Detail

High **Cross Site Scripting (Persistent)**

Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browser instance. A browser instance can be a standard web browser client, or a browser object embedded in a software product such as the browser within WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML/JavaScript, but may also extend to VBScript, ActiveX, Java, Flash, or any other browser-supported technology.

Figure 4 — ZAP alert summary table from the HTML report, grouped by risk level and number of instances.

Risk Level	Number of Alert Types	Examples
High	7	SQL Injection, XSS (Reflected & Persistent), Path Traversal, Open/External Redirect, Remote File Inclusion
Medium	4	Missing CSP header, Missing anti-clickjacking header, Absence of Anti-CSRF tokens, XSLT Injection
Low	3	Missing X-Content-Type-Options header, Cookie without SameSite attribute, Cross-domain JS inclusion
Informational	6	Suspicious code comments, session token identified, modern web app detected

Several High-risk findings from ZAP directly correspond to issues already flagged earlier in the pipeline by the SAST tool (Semgrep) — for example, the SQL Injection alerts on /login, /search, and /products match Semgrep's formatted-sql-string and tainted-sql-from-http-request findings in LoginServlet.java, SearchServlet.java, and ProductServlet.java. This overlap is expected and useful: SAST identifies the vulnerable pattern in the source code, while DAST confirms that the vulnerability is actually exploitable in the running application.

9. Conclusion and Next Steps

OWASP ZAP is now fully integrated into the CI/CD pipeline as an automated DAST stage that runs on every build, after deployment, without blocking the pipeline. Recommended next steps:

- Fix the High-risk findings (SQL Injection, XSS, Path Traversal, Open Redirect) identified in the current scan.

- Move from ZAP's Quick Scan to an authenticated full scan (zap-full-scan.py with a login context) to test pages that require a logged-in session.
- Introduce severity-based gating once the current known issues are triaged — for example, fail the build only on newly introduced Critical/High findings, while continuing to allow known, tracked issues through as UNSTABLE.
- Restrict the Security Group rule to the narrowest possible scope (already done here, limited to the Jenkins private IP) and review it periodically.